

A HANDS-ON SETUP HANDBOOK · FIRST EDITION

The Robotics Workstation Handbook

From a Windows PC to a GPU-ready Ubuntu machine with
Docker, ROS 2, Nav2, Gazebo and NVIDIA Isaac Sim

MOTION PLANNING

PERCEPTION

AUTONOMY

SIMULATION



*"Before you plan a path, fuse a sensor or drive a car,
you need a machine that works. This is the path I
wish I had on day one."*

MD ASIFUZZAMAN

© 2026 Md Asifuzzaman. All rights reserved.

The Robotics Workstation Handbook

A hands-on setup guide for aspiring robotics and autonomy engineers

First edition, October 2026

© 2026 Md Asifuzzaman. All rights reserved.

This handbook is free. You may read, download and share it free of charge in its complete, unmodified form, with credit to the author. You may not sell it or publish modified versions without the author's written permission.

Disclaimer. The steps in this handbook change disks, boot settings and system files. Follow them at your own risk, back up your data first, and check each project's official documentation before you install, because software versions change quickly. The author is not responsible for data loss or damage.

Trademarks. Ubuntu is a trademark of Canonical Ltd. NVIDIA, CUDA and Isaac Sim are trademarks of NVIDIA Corporation. ROS is a trademark of Open Robotics. Docker is a trademark of Docker, Inc. Windows, BitLocker and Visual Studio Code are trademarks of Microsoft Corporation. Other names belong to their owners. This handbook is independent and is not endorsed by any of these companies.

asif-ucchwas.github.io

Why I built this

When I decided to work in robotics and autonomy, the first real obstacle was not an algorithm. It was the machine.

Before I could test a single planner, I had to get Ubuntu, GPU drivers, Docker, ROS 2 and a simulator working together on one computer. The resources were everywhere, but scattered. Every video I found took a long time to reach the part where you actually type the commands, and every page assumed I had already done the step before it.

There is no real shortcut around this stack. Motion planning, perception, autonomous driving and simulation teams all expect you to work fluently on Linux, inside containers, with ROS 2 and a simulator. So I wrote down the exact path that worked on my own machine: every command in order, with a check at the end of each chapter so you know you are on track before you move on.

If you are serious about this field and willing to do the work, this handbook is for you. Follow it in order, read the output, and by the last page you will have a workstation ready for real robotics work.

Md Asifuzzaman

Beaumont, Texas

October 2026

Contents

How to use this handbook	5
Who needs this setup	7
Part I Before you begin	9
01 What you end up with	10
02 Before you start	12
03 The method	13
Part II The operating system	15
04 Prepare Windows	16
05 Install Ubuntu 24.04	17
06 First boot, update and base tools	20
07 Backups with Timeshift on a USB stick	22
Part III Everyday tools	25
08 Git and GitHub over SSH	26
09 VS Code and extensions	28
10 Docker and the NVIDIA Container Toolkit	30
Part IV Robotics	32
11 ROS 2 Jazzy, Nav2, TurtleBot3 and Gazebo	33
12 Workspace and first simulation test	35
13 Isaac Sim 6.1	38
Part V Polish and troubleshooting	41
14 Tidy the boot menu and speed up startup	42
15 Optional tools and Python environments	47
16 Troubleshooting and final checklist	49
Glossary	51
Sources	53

How to use this handbook

Sixteen chapters, one path. Work through them in order and run the check at the end of each chapter before you start the next.

What each chapter gives you

Every chapter opens with a short goal, so you always know what finished looks like:

DONE WHEN the test at the end of the chapter passes.

NOTE

Background that helps you understand a step.

WARNING

A step that can erase data or lock you out if done wrong.

✓ BEFORE YOU TURN THE PAGE

The test that proves the chapter worked.

To see which chapters matter most for your target role, turn to [Who needs this setup](#).

Copying commands from this PDF

Every command sits in a dark terminal box like the one below. The text is real, selectable text, and every line is short enough to paste cleanly.



1. Open the PDF in a viewer that lets you select text, such as your web browser, Document Viewer, Okular or Adobe Reader.
2. Drag from the first character of the box to the last character, then press **Ctrl + C**. Copy only the commands, not the title bar.
3. Click in the Ubuntu terminal and paste with **Ctrl + Shift + V**, or right-click and choose **Paste**.
4. Replace any placeholder such as `YOUR_NAME` with your own value.
5. Press **Enter** and read the output before you copy the next box.

TWO GOOD HABITS

A line that ends with a backslash `\` continues on the next line. Copy the whole box and it runs as one command. If a pasted command fails, compare it with the page; some viewers add or drop a space.

WHY THERE IS NO COPY BUTTON

PDF readers do not support copy buttons reliably, so this handbook keeps every command as plain text you can select. The online edition can add one-click copy buttons.

Who needs this setup

Every role starts from the same foundation: Chapters 1 to 8 and 16. Robotics software, motion planning, autonomy, perception and controls engineers then use nearly every remaining chapter, while simulation engineers lean most on Docker and Isaac Sim.

ROLE	WHAT THEY DO	CHAPTERS
Motion Planning	Designs path and trajectory planners that move robots and vehicles safely.	14 of 16
Autonomy / AV	Brings planning, perception and control together into self-driving systems.	15 of 16
Perception	Turns camera, LiDAR and radar data into an understanding of the world.	14 of 16
Simulation	Builds virtual worlds and sensor models to test robots before hardware.	11 of 16
Robotics Software	Builds the robot's software: nodes, tools, drivers and integration.	15 of 16
Controls	Makes the robot follow the planned motion accurately and stably.	14 of 16

Chapters by role

A dot means the chapter is core for that role. Chapters with a dot in every column are the shared foundation everyone needs.

CHAPTER	MOTION PLANNING	AUTONOMY/ AV	PERCEPTION	SIMULATION	ROBOTICS SOFTWARE	CONTROLS
01 What you end up with	●	●	●	●	●	●
02 Before you start	●	●	●	●	●	●
03 The method	●	●	●	●	●	●
04 Prepare Windows	●	●	●	●	●	●
05 Install Ubuntu 24.04	●	●	●	●	●	●
06 First boot, update and base tools	●	●	●	●	●	●
07 Backups with Timeshift on a USB stick	●	●	●	●	●	●
08 Git and GitHub over SSH	●	●	●	●	●	●
09 VS Code and extensions	●	●	●		●	●
10 Docker and the NVIDIA Container Toolkit		●	●	●	●	
11 ROS 2 Jazzy, Nav2, TurtleBot3 and Gazebo	●	●	●		●	●
12 Workspace and first simulation test	●	●			●	●
13 Isaac Sim 6.1	●	●	●	●		
14 Tidy the boot menu and speed up startup		●			●	●
15 Optional tools and Python environments	●		●		●	●
16 Troubleshooting and final checklist	●	●	●	●	●	●

PART I

Before you begin

What you will build, what you need, and how to work through this handbook.

01 What you end up with

02 Before you start

03 The method

01

PART I · BEFORE YOU BEGIN

What you end up with

DONE WHEN you know which tools you will install and why each one is there.

This handbook turns a Windows PC with an NVIDIA RTX GPU into a dual-boot Ubuntu 24.04 development machine with Docker, ROS 2 Jazzy, Nav2 with Gazebo, and Isaac Sim 6.1. It was written from a real install on one machine and is meant to be followed in order.

COMPONENT	VERSION USED	PURPOSE	HOW IT IS INSTALLED
Ubuntu Desktop	24.04 LTS (24.04.5.1)	Operating system	Rufus USB installer, manual partitions
NVIDIA driver	595.x (CUDA 13.2)	GPU support	Ubuntu installer, third-party drivers box
Timeshift	24.01	System snapshots	apt, backups on a USB stick
Google Chrome	Latest stable	Browser	.deb from google.com/chrome
Git and an SSH key	Git 2.43	Version control	apt, ssh-keygen
Visual Studio Code	Latest stable	Editor	Microsoft .deb
Docker Engine and Compose	Latest stable	Containers	Docker apt repository
NVIDIA Container Toolkit	Latest stable	GPU inside containers	NVIDIA apt repository
ROS 2	Jazzy Jalisco	Robotics framework	ROS 2 apt repository
Nav2, TurtleBot3, Gazebo (ros_gz)	Jazzy packages	Navigation and simulation	apt
Isaac Sim	6.1.0.0	NVIDIA robot simulator	pip in a Python 3.12 virtual environment
Tela GRUB theme	Latest from GitHub	Boot menu look	git clone and the project's install script

TESTED ON

Date	October 2026
Machine	Lenovo Legion Tower 7i Gen 10, Intel Core Ultra 7 265K
GPU	NVIDIA GeForce RTX 5080, 16 GB
Memory	32 GB DDR5
Software	Ubuntu 24.04.5, NVIDIA driver 595.x (CUDA 13.2), ROS 2 Jazzy, Isaac Sim 6.1.0.0

Versions change quickly. If your machine or versions differ, the steps usually still apply, but check each project’s official page before you install.

02

PART I · BEFORE YOU BEGIN

Before you start

DONE WHEN your hardware meets the requirements and your important files are backed up.

CHECK THESE POINTS FIRST

The Ubuntu installer erases whatever disk you point it at. Read every point below before you continue.

- **GPU:** an NVIDIA RTX card. Isaac Sim lists a GeForce RTX 4080 as its minimum and an RTX 5080 as the good tier; cards without RT cores (A100, H100) are not supported.
- **RAM and storage:** 32 GB RAM minimum (64 GB recommended) and an SSD with at least 50 GB free, ideally 500 GB or more.
- **A second internal drive is the safest layout.** This handbook installs Ubuntu on an empty second drive and leaves Windows alone. Shrinking a single Windows drive is a different procedure and is not covered here.
- **Use Ubuntu 24.04 LTS, not 26.04.** Isaac Sim's requirements page lists Ubuntu 22.04 and 24.04 only.
- **Back up** anything important on the drive you will erase.
- **BitLocker:** Windows 11 may have it on. Save the recovery key somewhere other than the PC before changing boot settings (Chapter 4).
- **Replace the placeholders.** Where a command contains `YOUR_NAME` , `YOUR_EMAIL` or `YOUR_USERNAME` , type your own values.

Source: Isaac Sim system requirements (see Sources at the end).

03

PART I · BEFORE YOU BEGIN

The method

DONE WHEN you understand the order of the chapters and the three habits below.

Work through the chapters in order, and run the check at the end of each one before you start the next, so a problem is caught while it is still small.

CH.	WHAT YOU DO	HOW YOU KNOW IT WORKED
4	Save the BitLocker recovery key and turn off Fast Startup	The key is stored off the PC
5	Make a USB installer, boot from it, partition the empty drive by hand	Ubuntu reaches its login screen
6	Update the system, confirm the GPU driver, install base tools	<code>nvidia-smi</code> lists your GPU
7	Set up Timeshift on a USB stick and take your first snapshot	A snapshot appears in Timeshift
8	Set your Git identity and add an SSH key	<code>ssh -T git@github.com</code> greets your username
9	Install VS Code and its extensions	It opens a folder
10	Install Docker Engine, then the NVIDIA Container Toolkit	A container prints the GPU table
11	Install ROS 2, Nav2, TurtleBot3 and Gazebo	The talker demo prints messages
12	Build a ROS 2 workspace and launch Nav2 in Gazebo	The robot reaches a goal
13	Install Isaac Sim in a Python virtual environment	The window opens and the log prints app ready
14	Install the Tela boot menu, keep a logo on screen, remove a network wait	Both systems start; <code>systemd-analyze</code> shows the new timing

Three habits make this go smoothly

1. **Copy commands exactly**, one block at a time, and read the output before the next one.

2. **If something fails, keep the last 15 lines** of the terminal output. They usually name the problem.
3. **Change one thing at a time.** If you do not know why something broke, you cannot undo it.

II

PART II

The operating system

Prepare Windows, install Ubuntu 24.04 next to it, and protect the new system with snapshots.

-
- 04 Prepare Windows
 - 05 Install Ubuntu 24.04
 - 06 First boot, update and base tools
 - 07 Backups with Timeshift on a USB stick
-

04

PART II · THE OPERATING SYSTEM

Prepare Windows

DONE WHEN the recovery key is stored off the PC and you know which drive is the empty one.

Do these three things in Windows before you boot the Ubuntu installer; they take about ten minutes.

1 Save your BitLocker recovery key

Open `aka.ms/myrecoverykey` in a browser, sign in with the Microsoft account used on the PC, and photograph or write down each 48-digit key. Keep the copy off the PC, for example on your phone or on paper.

WHY THIS MATTERS

Windows may ask for the key once after the boot setup changes, and without it you cannot get back in. Never paste the key into a chat or send it to anyone.

2 Identify the empty drive

Open File Explorer, then This PC, and note the size of the empty drive that will hold Ubuntu (for example 953 GB). You will recognise it in the installer by that size and by having no files.

3 Turn off Fast Startup

Press **Windows + R**, type `powercfg.cpl` and press Enter. Click *Choose what the power buttons do*, then *Change settings that are currently unavailable*, untick *Turn on fast startup*, and click *Save changes*. If the option is missing or already unticked, it is already off.

✓ BEFORE YOU TURN THE PAGE

You have the recovery key stored off the PC, and you know which drive is the empty one.

05

PART II · THE OPERATING SYSTEM

Install Ubuntu 24.04

DONE WHEN Ubuntu reaches the login screen and you can log in.

You need a USB stick of at least 8 GB, which will be erased, and about 45 minutes.

1 Download the ISO

In a browser, download Ubuntu Desktop 24.04 LTS for Intel/AMD 64-bit, about 6 GB. This install used `ubuntu-24.04.5.1-desktop-amd64.iso` from `releases.ubuntu.com/24.04` ; if that file name has moved on, take the newest 24.04.x from the Ubuntu download page.

2 Make the USB installer

Download Rufus from `rufus.ie` (the standard 64-bit .exe), plug in the stick and set:

SETTING	VALUE
Device	your USB stick
Boot selection	the Ubuntu .iso
Partition scheme	GPT
Target system	UEFI (non CSM)

Click **Start**, choose *ISO Image mode*, and accept the erase warning. Wait for the bar to say **READY**.

3 Boot from it

Restart and tap **F12** repeatedly (on other brands try F2, F10, F11 or Esc). Pick the entry for your USB stick (UEFI), then *Try or Install Ubuntu*.

4 Walk through the installer

Choose language, keyboard and network. Choose *Interactive installation* and *Default selection*. Tick both boxes for third-party software (graphics and Wi-Fi drivers, and media formats). The NVIDIA driver comes from this box.

5 Choose Manual installation on the disk screen

DO NOT USE THE DEFAULT

The option *Erase disk and install Ubuntu* is selected by default and can wipe the Windows disk. Do not use the default.

6 Partition the empty drive only

The plan is in the next section.

7 Create your account

Computer name: lowercase letters, numbers and hyphens only. Username: lowercase, no spaces. Pick a password you will remember. Confirm the time zone.

8 Review, then install

The summary must list only partitions of the empty drive. If it mentions the Windows drive, press **Back**. Click **Install**.

9 Finish

Click *Restart now*, remove the USB stick when asked, and press Enter. If Windows starts instead, restart, press F12 and choose the *ubuntu* entry.

Partition plan for a 1 TB empty drive

In the partitioning screen the drive names look like `nvme0n1` and `nvme1n1` . The Windows drive has several partitions, one of them labelled BitLocker. The empty one is the other drive. Never touch the Windows drive or the USB installer.

PARTITION	SIZE	FORMAT	MOUNT POINT
EFI boot	1 GB	FAT32 (shown as VFAT)	<code>/boot/efi</code>
Swap	16 GB	Swap	none
System	250 GB	Ext4	<code>/</code>
Home	all the rest	Ext4	<code>/home</code>

How to build it:

1. Click the empty drive’s row (the disk itself, not a partition), then *New partition table* and confirm.
2. Select the free space and click **+** once per partition, in the order above.

3. Set *Device for boot loader installation* to the same empty drive, not the Windows drive.
4. Click **Next**.

IF NEXT IS GREYED OUT

If **Next** stays greyed out, the installer has probably added its own small FAT32 partition. Delete your own EFI partition with the – button, select the installer’s FAT32 partition, click *Change*, and set its mount point to `/boot/efi`. Check that the boot loader device is still set. A leftover 1 GB of free space is harmless.

✓ BEFORE YOU TURN THE PAGE

Ubuntu reaches the login screen and you can log in.

06

PART II · THE OPERATING SYSTEM

First boot, update and base tools

DONE WHEN `git --version` prints a version number and `nvidia-smi` shows your GPU.

From here on you type commands in a terminal; open one with **Ctrl + Alt + T**. When `sudo` asks for your password, nothing appears as you type, which is normal. Press Enter when done.

1 Confirm the GPU driver

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
nvidia-smi
```

EXPECTED a table showing your GPU name, a driver version and a CUDA version. If it says `command not found`, the third-party drivers box was not ticked in the installer; install the driver with `sudo ubuntu-drivers install` and restart.

2 Update the system, then restart

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt update && sudo apt upgrade -y  
sudo reboot
```

If a purple screen asks which services to restart, press Enter.

3 Install the base tools

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt install -y git curl wget build-essential cmake \  
python3-pip python3-venv python3-dev htop tree
```

4 Install Google Chrome (optional)

Ubuntu ships Firefox; for Chrome, download the official package and install it.

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
cd /tmp
URL=https://dl.google.com/linux/direct
wget $URL/google-chrome-stable_current_amd64.deb
sudo apt install -y ./google-chrome-stable_current_amd64.deb
```

✓ BEFORE YOU TURN THE PAGE

`git --version` prints a version number, and `nvidia-smi` still shows your GPU.

07

PART II · THE OPERATING SYSTEM

Backups with Timeshift on a USB stick

DONE WHEN Timeshift lists a snapshot and its footer names `/dev/sda1` .

Timeshift saves restore points of the Ubuntu system, so a failed install can be rolled back instead of reinstalling Ubuntu. It does not back up your `/home` files by default, so keep projects on GitHub too. It comes now, before the big installs, so you can take a snapshot before each of them.

This chapter splits a 128 GB stick into a 64 GB backup partition and a general storage partition that Windows can also read.

WARNING

The commands below erase the whole stick. Run the first command and make sure you pick the right device; choosing your Ubuntu drive by mistake destroys your system.

1 Plug in the stick and find its device name

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,LABEL,MOUNTPOINTS
```

The stick is the disk whose size matches (about 114 GB for a 128 GB stick) and has no `/` or `/home` mountpoints. In this handbook it is `sda` ; if yours differs, change `sda` in every command below.

2 Partition and format it

This erases the stick.

```
sudo umount /dev/sda1
sudo parted /dev/sda --script mklabel gpt \
    mkpart BACKUP ext4 1MiB 64GiB mkpart STORAGE 64GiB 100%
sudo mkfs.ext4 -L BACKUP /dev/sda1
sudo apt install -y exfatprogs
sudo mkfs.exfat -L STORAGE /dev/sda2
lsblk -f /dev/sda
```

If `umount` says not mounted, carry on. If `mkfs.ext4` asks *Proceed anyway?* about an old filesystem, type `y`.

EXPECTED `sda1 ext4 BACKUP` and `sda2 exfat STORAGE`.

3 Install Timeshift and give your user the BACKUP partition

Unplug and replug the stick first so both partitions mount.

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt install -y timeshift
sudo chown $USER:$USER /media/$USER/BACKUP
```

4 Set up Timeshift

Open it from the Super (Windows) key menu and follow the wizard:

SCREEN	CHOICE
Snapshot type	RSYNC
Location	the BACKUP partition (ext4, about 64 GB), not an internal drive
Schedule	tick Weekly, keep 3, or leave all unticked to snapshot by hand. Scheduled snapshots only run while the stick is plugged in.
User home directories	leave the default, which excludes /home

Click **Finish**, then **Create**.

5 Remove the stick safely

Close Timeshift and the Files window, then unmount both partitions. Eject only works once both are unmounted.

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo umount -A /dev/sda1 /dev/sda2
lsblk /dev/sda
```

When no mountpoints show for `sda1` and `sda2`, unplug it.

WHEN TO TAKE A SNAPSHOT

Take a snapshot before the Docker (Chapter 10), ROS 2 (Chapter 11) and Isaac Sim (Chapter 13) installs, and before any large upgrade. To restore, open Timeshift, pick a snapshot and click **Restore**.

✓ BEFORE YOU TURN THE PAGE

Timeshift's main window lists a snapshot, and its footer names `/dev/sda1` .

III

PART III

Everyday tools

Git over SSH, VS Code, and Docker with GPU access: the tools every robotics team uses daily.

08 Git and GitHub over SSH

09 VS Code and extensions

10 Docker and the NVIDIA Container Toolkit

08

PART III · EVERYDAY TOOLS

Git and GitHub over SSH

DONE WHEN a clone finishes and the repository's files appear in `~/projects`.

An SSH key lets you push and pull without typing a password. Replace `YOUR_NAME` and `YOUR_EMAIL` with your own values; the email must be one listed on your GitHub account.

1 Set your identity **TERMINAL**

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
git config --global user.name "YOUR_NAME"
git config --global user.email "YOUR_EMAIL"
git config --global init.defaultBranch main
```

2 Create a key

Press Enter to accept the file location. A passphrase is optional.

 **TERMINAL**

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
ssh-keygen -t ed25519 -C "YOUR_EMAIL" -f ~/.ssh/id_ed25519_github
```

3 Tell SSH to use that key for GitHub **TERMINAL**

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
cat >> ~/.ssh/config <<'EOF'
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_github
  IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
```

4 Print the public key and copy the whole line **TERMINAL**

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
cat ~/.ssh/id_ed25519_github.pub
```

5 Add it to GitHub

In a browser, log in and open `github.com/settings/keys`, click **New SSH key**, give it a title such as *Ubuntu PC*, paste the line, and click **Add SSH key**.

KEEP THE PRIVATE KEY PRIVATE

Share only the `.pub` file. Never share the file without `.pub`; it is your private key.

6 Test it

Type `yes` if it asks about the fingerprint.

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
ssh -T git@github.com
```

EXPECTED Hi YOUR_USERNAME! You've successfully authenticated, but GitHub does not provide shell access.

7 Clone a repository

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
mkdir -p ~/projects && cd ~/projects  
git clone git@github.com:YOUR_USERNAME/YOUR_REPO.git
```

✓ BEFORE YOU TURN THE PAGE

The clone finishes and the repository's files appear in `~/projects`.

09

PART III · EVERYDAY TOOLS

VS Code and extensions

DONE WHEN a folder opens in VS Code and the Restricted Mode bar is gone.

VS Code handles Python, C++, CMake, ROS 2 packages, Dockerfiles and Git in one editor, which is why it comes before the heavier tools.

1 Download and install it

```
cd /tmp
wget -O code.deb \
  "https://code.visualstudio.com/sha/download?build=stable&os=linux-deb-x64"
sudo apt install -y ./code.deb
```

When a purple screen asks *Add Microsoft apt repository for Visual Studio Code?*, choose **Yes**. That lets Ubuntu update VS Code together with everything else.

2 Install the extensions

```
code --install-extension ms-python.python
code --install-extension ms-vscode.cpptools
code --install-extension ms-vscode.cmake-tools
code --install-extension ms-iot.vscode-ros
code --install-extension ms-azuretools.vscode-docker
```

Each prints *was successfully installed*. Deprecation warnings about `url.parse()` are harmless.

3 Open a project folder

```
code ~/projects/YOUR_REPO
```

4 Answer the pop-ups

POP-UP	WHAT TO DO
Sign in for GitHub Copilot	Choose <i>Continue without Signing In</i> . You can sign in later.
Make It Yours theme page	Pick any theme and click <i>Get Started</i> .
Restricted Mode bar	Click <i>Manage</i> , then <i>Trust Folder & Contributors</i> , for folders you wrote yourself. For code you downloaded from others, read it before trusting it.
Pre-release C/C++ extension	Click <i>No</i> .

✓ BEFORE YOU TURN THE PAGE

The folder opens in VS Code, the yellow Restricted Mode bar is gone, and the file tree shows your repository.

PART III · EVERYDAY TOOLS

10 Docker and the NVIDIA Container Toolkit

DONE WHEN a container prints the same GPU table you saw on the host.

This chapter installs Docker Engine from Docker's own repository, then lets containers see your GPU. Take a Timeshift snapshot first (Chapter 7). Run each block, then read the output before the next.

WHY TRUST THESE SOURCES?

Several commands below add a signing key and a package source. Add these only from each project's official documentation (here Docker and NVIDIA), because a package source can install anything on your system.

1 Add Docker's repository

TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
CODENAME=$(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $CODENAME stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

2 Install Docker Engine and Compose

TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

3 Run Docker without sudo

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo usermod -aG docker $USER
newgrp docker
docker run hello-world
```

EXPECTED Hello from Docker! . `newgrp` applies only to the current terminal; after you next log out and in, every terminal works.

4 Add the NVIDIA Container Toolkit repository

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
  sudo gpg --dearmor \
  -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
```

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
KEYRING=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
LIST=https://nvidia.github.io/libnvidia-container/stable/deb
curl -s -L $LIST/nvidia-container-toolkit.list | \
  sed "s#deb https://#deb [signed-by=$KEYRING] https://#g" | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

5 Install it and connect it to Docker

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt update
sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

6 Test the GPU inside a container

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
docker run --rm --gpus all ubuntu nvidia-smi
```

✓ BEFORE YOU TURN THE PAGE

The container prints the same GPU table you saw on the host.

IV

PART IV

Robotics

ROS 2 Jazzy, Nav2 in Gazebo, and NVIDIA Isaac Sim, tested end to end.

11 ROS 2 Jazzy, Nav2, TurtleBot3 and Gazebo

12 Workspace and first simulation test

13 Isaac Sim 6.1

PART IV · ROBOTICS

11 ROS 2 Jazzy, Nav2, TurtleBot3 and Gazebo

DONE WHEN `ros2 pkg list` shows `nav2_bringup` and `turtlebot3_gazebo`.

Each ROS 2 release is built for one Ubuntu version, and Jazzy Jalisco is the release for Ubuntu 24.04. The desktop install is about 3 GB and takes several minutes. Take a Timeshift snapshot first (Chapter 7).

1 Enable the universe repository and add the ROS 2 source

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt install -y software-properties-common
sudo add-apt-repository -y universe
```

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
REPO=https://api.github.com/repos/ros-infrastructure/ros-apt-source
export ROS_APT_SOURCE_VERSION=$(curl -s $REPO/releases/latest \
  | grep -F "tag_name" | awk -F\" '{print $4}')
CODENAME=$(. /etc/os-release && echo ${UBUNTU_CODENAME:-${VERSION_CODENAME}})
DL=https://github.com/ros-infrastructure/ros-apt-source/releases/download
curl -L -o /tmp/ros2-apt-source.deb \
  "$DL/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.\
  ${CODENAME}_all.deb"
sudo dpkg -i /tmp/ros2-apt-source.deb
```

2 Install ROS 2 and the developer tools

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt update
sudo apt install -y ros-jazzy-desktop ros-dev-tools
```

3 Load ROS 2 in every new terminal

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

4 Test it

Press **Ctrl + C** to stop.

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
printenv ROS_DISTRO
ros2 run demo_nodes_cpp talker
```

EXPECTED `jazzy`, then lines such as `Publishing: 'Hello World: 1'.`

5 Install Nav2, TurtleBot3 and the Gazebo bridge

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt install -y ros-jazzy-navigation2 ros-jazzy-nav2-bringup \
  ros-jazzy-turtlebot3 ros-jazzy-turtlebot3-gazebo ros-jazzy-ros-gz
```

6 Set the robot model and initialise rosdep

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
echo "export TURTLEBOT3_MODEL=waffle" >> ~/.bashrc
source ~/.bashrc
sudo rosdep init
rosdep update
```

If `rosdep init` says the file already exists, ignore it and run `rosdep update`.

✓ BEFORE YOU TURN THE PAGE

`ros2 pkg list | grep -E "nav2_bringup|turtlebot3_gazebo"` prints both names.

12

PART IV · ROBOTICS

Workspace and first simulation test

DONE WHEN Gazebo and RViz are open and the terminal ends with `Goal succeeded`.

A ROS 2 workspace is a folder with a `src` directory where your packages live. This chapter builds one and launches Nav2 with a TurtleBot3 in Gazebo to prove the stack works.

1 Create the workspace and add your package

Replace the repository name with your own ROS 2 package, or skip the clone to test the simulation alone.

TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
git clone git@github.com:YOUR_USERNAME/YOUR_ROS2_PACKAGE_REPO.git
```

2 Install Python dependencies with apt, not pip

Ubuntu 24.04 refuses system-wide `pip install` to protect system tools, and the apt versions match ROS 2. Use the names your package needs; these two are examples.

TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt install -y python3-numpy python3-matplotlib
```

3 Let rosdep fetch what package.xml asks for, then build

TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
cd ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
```

EXPECTED Summary: 1 package finished (more if you added several). Warnings about `setup.py install` being deprecated are harmless.

4 Load the workspace in every terminal and list your nodes

```
select → Ctrl+C → paste in terminal with Ctrl+Shift+V

source install/setup.bash
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
ros2 pkg executables YOUR_PACKAGE_NAME
```

5 Launch the simulation

Run this in its own terminal and leave it open. The first launch takes a minute or two, then Gazebo and RViz open.

```
select → Ctrl+C → paste in terminal with Ctrl+Shift+V

ros2 launch nav2_bringup tb3_simulation_launch.py headless:=False
```

6 Tell Nav2 where the robot starts

In a second terminal, publish the TurtleBot3 world's default start pose.

```
select → Ctrl+C → paste in terminal with Ctrl+Shift+V

ros2 topic pub --once /initialpose \
  geometry_msgs/msg/PoseWithCovarianceStamped \
  "{header: {frame_id: 'map'}, pose: {pose: {position: \
  {x: -2.0, y: -0.5, z: 0.0}, orientation: {w: 1.0}}}}"
```

In RViz the scan lines should now line up with the walls, and the red Global Status error should turn Ok. You can set the pose by hand instead: click *2D Pose Estimate* in the toolbar, click where the robot is, hold, drag the way it faces and release.

7 Send a goal

In RViz click *Nav2 Goal*, click a free spot on the map, hold, drag and release. The robot drives there, and the launch terminal prints `Goal succeeded`.

TWO DETAILS WORTH KNOWING

The Nav2 Goal button calls an action, so it does not publish on the `/goal_pose` topic. To feed your own planner node, send the goal yourself with `ros2 topic pub --once /goal_pose geometry_msgs/msg/PoseStamped ...`, using the topic that node subscribes to. The warning *Failed to transform initial pose in time* in the first seconds is a harmless timing message.

✓ BEFORE YOU TURN THE PAGE

Gazebo and RViz are open, and the terminal ends with `Goal succeeded`.

13

PART IV · ROBOTICS

Isaac Sim 6.1

DONE WHEN the window opens, the log prints `app ready` and the sphere falls onto the plane.

Isaac Sim 6.1 installs from NVIDIA's pip index into a Python 3.12 virtual environment, and Ubuntu 24.04 already ships Python 3.12. The download is tens of GB, so start it when you can leave the machine alone.

Pip installs need glibc 2.35 or newer, which Ubuntu 24.04 has, and they are started with the `isaacsim` command rather than an `isaac-sim.sh` launcher.

WHAT IS A VIRTUAL ENVIRONMENT?

A virtual environment is a private folder of Python packages, so Isaac Sim's many pinned versions cannot clash with ROS 2 or the system tools.

1 Take a snapshot

Open Timeshift and create a snapshot (Chapter 7) before this long install.

2 Check the Python version

```
python3 --version
```

EXPECTED Python 3.12.x .

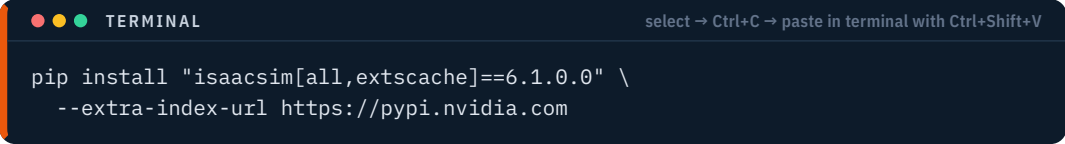
3 Create and activate the environment

The prompt gains a `(isaacsim)` prefix while it is active.

```
mkdir -p ~/venvs
python3 -m venv ~/venvs/isaacsim
source ~/venvs/isaacsim/bin/activate
pip install --upgrade pip
```

4 Install Isaac Sim

This is the long step.

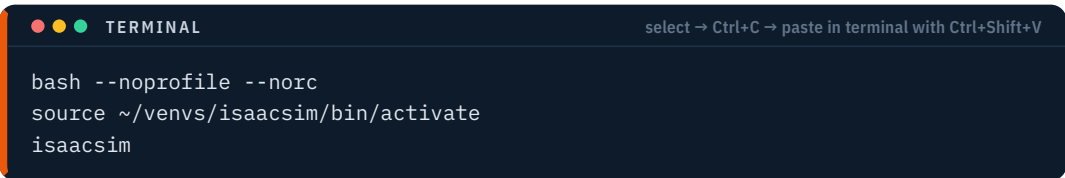


```
pip install "isaacsim[all,extscache]==6.1.0.0" \
--extra-index-url https://pypi.nvidia.com
```

It ends with *Successfully installed* and a long package list. A red-looking pip message about `generate-parameter-library-py` needing `typeguard` is a ROS 2 helper complaint and does not affect Isaac Sim; `sudo apt install python3-typeguard` silences it.

5 Start it in a clean shell

ROS 2 is loaded by your normal terminal, and it can interfere with Isaac Sim's own libraries, so open a plain shell first. Your normal shell is the Bash program inside the terminal window, and `--noprofile --norc` starts a second one that skips the ROS setup lines.



```
bash --noprofile --norc
source ~/venvs/isaacsim/bin/activate
isaacsim
```

The first launch asks you to accept NVIDIA's licence: type `Yes`. It then builds shader caches and loads extensions. That took about 2.5 minutes on the reference machine, and it can take longer on a slow connection, so do not close the terminal.

6 Read the result

Good signs: the window title reads *Isaac Sim Full 6.1.0*, the terminal ends with *Isaac Sim Full App is loaded* and `app ready`, and the viewport shows an FPS figure and your GPU name. Orange warnings such as `mdl_list_cache is not complete` are normal on first run, and the log also shows `isaacsim.ros2.bridge` starting, which is the link to ROS 2.

7 Quick physics test

1. Menu *Create > Shape > Sphere*, then *Create > Physics > Ground Plane*.
2. In the Stage panel click *Sphere*. In the Property panel click *Add > Physics > Rigid Body with Colliders Preset*.
3. Set *Translate Z* to 3 under Transform, so the sphere floats above the ground (Z is up).
4. Click the **Play** triangle in the left toolbar. The sphere should fall and rest on the plane.

TIP

Switch to the Select arrow in the left toolbar first, because the Move gizmo drags objects when you click the viewport. If you move something by accident, press **Ctrl + Z**. The sphere will not bounce unless you give it a physics material with a restitution value.

8 When you finish

Close the Isaac Sim window, then type `exit` to leave the clean shell.

✓ BEFORE YOU TURN THE PAGE

The window opens, the log prints `app ready`, and the sphere falls onto the plane.

V

PART V

Polish and troubleshooting

A clean boot menu, useful extra tools, and fixes for the problems people hit most.

14 Tidy the boot menu and speed up startup

15 Optional tools and Python environments

16 Troubleshooting and final checklist

PART V · POLISH AND TROUBLESHOOTING

14 Tidy the boot menu and speed up startup

DONE WHEN the menu lists Ubuntu and Windows and both start, and `systemd-analyze` shows the new timing.

A fresh install shows a plain boot menu and can sit on a black screen for several seconds. This chapter gives the menu a clean look, hides two lines you rarely need, keeps a logo on screen while Ubuntu starts, and removes one avoidable wait. All of it is optional, every change can be undone, and one restart at the end shows the result.

The changes are made on the Ubuntu drive, and none of them writes to the Windows drive. Take a Timeshift snapshot first (Chapter 7).

1 Measure first and keep a copy of the GRUB settings

`systemd-analyze` splits a boot into four parts, and `blame` lists the slowest start-up services. Saving the result gives you something to compare with later. The last command copies the file that holds the boot menu settings, so you can undo.

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
systemd-analyze | tee ~/boot-before.txt
systemd-analyze blame | head -10 | tee -a ~/boot-before.txt
sudo cp /etc/default/grub /etc/default/grub.bak
```

This is how the report read on the reference machine before any change:

PART	BEFORE	WHAT IT IS
Firmware	9.5 s	The PC's own start-up, before Ubuntu is involved. This chapter does not change it.
Loader	9.5 s	GRUB, the boot menu.
Kernel	1.8 s	Linux itself starting.
User space	9.5 s	Services and the desktop session. This is the part step 6 shortens.
Total	30.3 s	

HOW TO UNDO

To undo the menu settings later, copy the backup back and rebuild the menu. Each time you change the file again, save a copy under a new name such as `grub.bak2`, so the earlier copy survives.

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo cp /etc/default/grub.bak /etc/default/grub
sudo update-grub
```

2 Install the Tela theme

Tela is one of four themes in the project `vinceliuice/grub2-themes` (tela, vimix, stylish and whitesur). Its installer also rebuilds the boot menu for you. Pick the `-s` value that matches your monitor: `1080p` (1920 x 1080, the default), `2k` (2560 x 1440), `4k` (3840 x 2160), `ultra` wide or `ultrawide2k`.

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
cd ~ && git clone https://github.com/vinceliuice/grub2-themes.git
cd ~/grub2-themes
sudo ./install.sh -t tela -s 2k
```

EXPECTED a line that reads `Found Windows Boot Manager`, then `All done!`. The installer also adds a UEFI Firmware Settings line to the menu, which step 3 hides.

NOTE

Run the installer from inside the folder. If you see `command not found` and your prompt ends in `~$`, run `cd ~/grub2-themes` and try again. If the menu later looks cut off or blurry, reinstall with your exact size, for example `sudo ./install.sh -t tela -c 2560x1440`. To remove the theme, run `sudo ./install.sh -r -t tela`.

3 Hide the Memory test and UEFI Firmware Settings lines

The menu is built only from the scripts in `/etc/grub.d` that are marked executable, so taking the mark away hides a line without deleting anything. The Memory test lines come from `20_memtest86+`, and the UEFI Firmware Settings line comes from `30_uefi-firmware`.

● ● ● TERMINAL

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo chmod -x /etc/grub.d/20_memtest86+ /etc/grub.d/30_uefi-firmware
```

If a name differs on your machine, list them with `ls /etc/grub.d`. To bring a line back, run `sudo chmod +x` on the same file and rebuild the menu in step 7. You can still open the BIOS setup with the key your PC shows at power-on.

4 Set the menu time and keep the screen sharp

Ubuntu's default setting, `GRUB_TIMEOUT_STYLE=hidden`, skips the menu and starts Ubuntu straight away. On a dual-boot machine you would then have to press Esc or hold Shift to reach Windows. This block makes the menu always show and sets how many seconds it waits; 7 is enough time to choose without a long wait, and you can change the number.

```

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

sudo sed -i 's/^GRUB_TIMEOUT=.*GRUB_TIMEOUT=7/' /etc/default/grub
sudo sed -i 's/^GRUB_TIMEOUT_STYLE=.*GRUB_TIMEOUT_STYLE=menu/' \
/etc/default/grub
sudo sed -i '/^GRUB_GFXPAYLOAD_LINUX=/d' /etc/default/grub
echo 'GRUB_GFXPAYLOAD_LINUX=keep' | sudo tee -a /etc/default/grub
KEYS=' TIMEOUT|TIMEOUT_STYLE|CMDLINE_LINUX_DEFAULT|GFXMODE|GFXPAYLOAD_LINUX'
grep -E "^GRUB_($KEYS)" /etc/default/grub

```

`GRUB_GFXPAYLOAD_LINUX=keep` keeps the screen resolution GRUB already chose instead of switching it when Ubuntu starts, which can make the logo screen look soft.

EXPECTED from the last command: `GRUB_TIMEOUT_STYLE=menu`, `GRUB_TIMEOUT=7`, `GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"`, a `GRUB_GFXMODE` line that the Tela installer set for your screen size, and `GRUB_GFXPAYLOAD_LINUX=keep`.

KEEP SPLASH

Keep `splash` in that line. On the reference machine it was removed for a while, and the screen stayed black for about 13 seconds between the menu and the login screen. If you ever removed it, put it back with:

```

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

sudo sed -i \
'/^GRUB_CMDLINE_LINUX_DEFAULT=/ s/"quiet *"/"quiet splash"/' \
/etc/default/grub

```

5 Choose the boot logo (optional)

The screen between the menu and the login screen is drawn by Plymouth. Ubuntu's default theme, `bgrt`, shows the PC maker's logo together with the Ubuntu logo and a spinner. The `spinner` theme shows only the Ubuntu logo and a spinner.

 **TERMINAL**

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo update-alternatives --config default.plymouth
```

A numbered list appears. Type the number of the line that ends in `spinner.plymouth` and press Enter. The new theme takes effect after step 7. To go back, run the same command and choose the `bgrt` line marked *auto mode*.

6 Remove the network wait (optional)

A desktop rarely needs to wait for the network before it shows the login screen. On the reference machine `NetworkManager-wait-online.service` took 5.7 s, and `docker.service` waited behind it. Look at the list you saved in step 1; if that service is not near the top, skip this step.

 **TERMINAL**

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo systemctl disable NetworkManager-wait-online.service
```

Check it in a separate command:

 **TERMINAL**

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
systemctl is-enabled NetworkManager-wait-online.service
```

EXPECTED `disabled.`

NOTE

Run `systemctl is-enabled` on its own. It exits with an error code when the answer is disabled, so anything chained after it with `&&` never runs. If you mount network drives at boot, or a service must have the network the moment it starts, turn the wait back on with `sudo systemctl enable NetworkManager-wait-online.service`.

7 Apply the changes and restart once

This rebuilds the boot menu and the start-up files in one go. If you skipped step 5, leave out the second command.

 **TERMINAL**

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo update-grub && sudo update-initramfs -u
```

EXPECTED Found theme: `/usr/share/grub/themes/tela/theme.txt`, your kernel lines, `For Windows Boot Manager` and `done`, then a line that starts `update-initramfs: Generating`. Then restart:

 **TERMINAL**

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo reboot
```

DRIVE NAMES CAN CHANGE

Drive names can swap between boots. On the reference machine Windows was reported on `nvme1n1p1` in one run and `nvme0n1p1` in the next. The menu finds Windows by its ID, so it keeps working, but run `lsblk` and read it before any command that names a disk, as Chapter 7 does for the USB stick.

8 Check the result and measure again

After the restart you should see, in this order: the Tela menu listing Ubuntu, Advanced options for Ubuntu and Windows Boot Manager with a countdown; a logo and spinner instead of a black screen; and the login screen. Restart once more and choose Windows to confirm it still starts. Windows may ask for the BitLocker key once after a boot change (Chapter 4).

Then run the report again and compare it with the saved one:

 **TERMINAL**

select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
systemd-analyze  
systemd-analyze blame | head -10  
cat ~/boot-before.txt
```

The user space figure should be shorter if you removed the network wait. Your numbers will differ from the reference machine, so compare your own before and after.

✓ BEFORE YOU TURN THE PAGE

The menu lists Ubuntu and Windows and both start, a logo shows while Ubuntu starts, and `systemd-analyze` shows the new timing next to the saved one.

PART V · POLISH AND TROUBLESHOOTING

15 Optional tools and Python environments

DONE WHEN the tools you chose are installed and the C++ example prints its message.

None of this is required for the earlier chapters, but each item saves time later.

Handy command-line tools

● ● ● TERMINAL select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
sudo apt install -y gdb valgrind clang-format tmux nvidia-top python3-pytest
```

TOOL	WHY YOU WANT IT
<code>gdb</code> , <code>valgrind</code>	Debug C++ and catch memory bugs.
<code>clang-format</code>	Formats C++ code automatically.
<code>tmux</code>	Splits one terminal into panes, useful when a simulation and several nodes run at once.
<code>nvidia-top</code>	Shows live GPU and VRAM use while Isaac Sim or Gazebo runs.
<code>python3-pytest</code>	Runs tests for your ROS 2 Python packages.

Compile C++ without an IDE

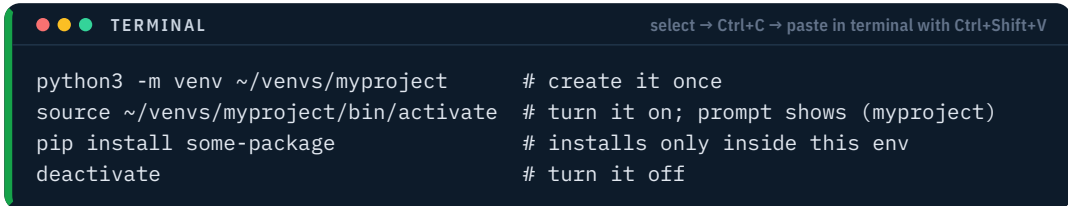
The compiler arrived with `build-essential` in Chapter 6.

● ● ● TERMINAL select → Ctrl+C → paste in terminal with Ctrl+Shift+V

```
mkdir -p ~/cpp-practice && cd ~/cpp-practice
cat > hello.cpp <<'EOF'
#include <iostream>
int main() {
    std::cout << "Hello from Ubuntu\n";
    return 0;
}
EOF
g++ -std=c++20 -Wall -Wextra hello.cpp -o hello && ./hello
```

Python virtual environments in four commands

Ubuntu 24.04 blocks system-wide `pip install`, so any pip-installed project gets its own environment. The folder name is yours to choose.

A terminal window with a dark background and light text. The title bar shows three colored circles (red, yellow, green) and the word "TERMINAL". On the right side of the title bar, it says "select → Ctrl+C → paste in terminal with Ctrl+Shift+V". The terminal contains four lines of code with comments:

```
python3 -m venv ~/venvs/myproject      # create it once
source ~/venvs/myproject/bin/activate  # turn it on; prompt shows (myproject)
pip install some-package               # installs only inside this env
deactivate                             # turn it off
```

ROS 2 does not use a virtual environment; it installs through apt on the system Python. Isaac Sim needs its own, as in Chapter 13.

Screenshots

Shift + PrtSc drags a box and saves it to Pictures > Screenshots. **Ctrl + Shift + PrtSc** copies it to the clipboard to paste into a chat.

Later, if you need them

LibreOffice for local Office files (`sudo snap install libreoffice`), Overleaf in the browser for LaTeX papers, and Isaac Lab for reinforcement learning on top of Isaac Sim, which is a separate and larger setup.

16

PART V · POLISH AND TROUBLESHOOTING

Troubleshooting and final checklist

DONE WHEN every box in the final checklist is ticked.

Most problems on this path come from a skipped step, so check the table before anything else.

PROBLEM	LIKELY CAUSE	FIX
<code>git: command not found</code>	Base tools not installed	Run the apt install line in Chapter 6
<code>nvidia-smi: command not found</code>	NVIDIA driver missing	<code>sudo ubuntu-drivers install</code> , then <code>sudo reboot</code>
Installer Next is greyed out	No valid EFI partition, or no boot loader device	Follow the note under the partition plan in Chapter 5
Windows asks for a BitLocker key	The boot setup changed	Type the key you saved in Chapter 4. Windows Hello may also ask you to set your PIN again.
USB stick will not eject	One partition is still mounted	<code>sudo umount -A /dev/sda1 /dev/sda2</code> and close Timeshift (Chapter 7)
Docker: permission denied	Your user is not in the docker group	<code>sudo usermod -aG docker \$USER</code> , then log out and in
Container cannot see the GPU	Toolkit not connected to Docker	<code>sudo nvidia-ctk runtime configure --runtime=docker</code> , then <code>sudo systemctl restart docker</code>
<code>ros2: command not found</code>	ROS 2 is not loaded in this terminal	<code>source /opt/ros/jazzy/setup.bash</code> , or open a new terminal
RViz map and robot do not line up	Initial pose not set	Publish <code>/initialpose</code> or use 2D Pose Estimate (Chapter 12)
Nav2 Goal does not trigger your own node	The button calls an action, not a topic	Publish the goal on the topic your node subscribes to
<code>isaacsim: command not found</code>	The virtual environment is not active	<code>source ~/venvs/isaacsim/bin/activate</code>
Isaac Sim looks frozen at first launch	Shader caches are being built	Wait several minutes and keep the terminal open
<code>sudo: ./install.sh: command not found</code>	You are not inside the grub2-themes folder	<code>cd ~/grub2-themes</code> , then run the installer again (Chapter 14)

PROBLEM	LIKELY CAUSE	FIX
Boot menu does not show	GRUB_TIMEOUT_STYLE is hidden	Set it to menu in <code>/etc/default/grub</code> , then <code>sudo update-grub</code> (Chapter 14)
Black screen for many seconds at start	splash was removed from the boot options	Set <code>"quiet splash"</code> , then <code>sudo update-grub</code> (Chapter 14)
Start-up waits for the network	NetworkManager-wait-online is slow	<code>sudo systemctl disable NetworkManager-wait-online.service</code> (Chapter 14)

Final checklist

- ☐
BitLocker recovery key stored off the PC
- ☐
Ubuntu boots, and `nvidia-smi` shows the GPU
- ☐
First Timeshift snapshot taken
- ☐
`ssh -T git@github.com` greets your username
- ☐
Docker prints the GPU table inside a container
- ☐
`ros2 run demo_nodes_cpp talker` prints messages
- ☐
Nav2 sends the robot to a goal in Gazebo
- ☐
Isaac Sim prints `app ready` and the sphere falls
- ☐
The boot menu lists Ubuntu and Windows, and both start

When every box is ticked, your workstation is ready for real robotics work: writing planners, training perception models, and testing autonomy in simulation.

Glossary

Short, plain-language meanings of the technical words used in this handbook.

apt

Ubuntu's package manager. `sudo apt install NAME` downloads and installs software from trusted repositories.

BitLocker

Windows disk encryption. Changing boot settings can make Windows ask for its 48-digit recovery key.

colcon

The build tool for ROS 2 workspaces. `colcon build` compiles every package in `src`.

Container

A packaged, isolated environment that runs an app with all its dependencies. Docker creates and runs containers from images.

CUDA

NVIDIA's platform for running computations on the GPU, used by perception models and simulators.

EFI partition

A small FAT32 partition, mounted at `/boot/efi`, that holds the files the PC needs to start an operating system.

ext4 / exFAT

File system formats. ext4 is Linux's standard; exFAT can be read by both Linux and Windows.

Gazebo

An open-source robot simulator with physics and sensors, connected to ROS 2 through `ros_gz`.

GRUB

The boot menu that lets you choose Ubuntu or Windows when the computer starts.

Isaac Sim

NVIDIA's GPU-based robot simulator with realistic rendering, physics and synthetic sensor data.

Nav2

The ROS 2 navigation stack: maps, localization, global planners and local controllers.

Node, topic, action

ROS 2 basics. A node is a running program; topics stream messages between nodes; actions handle long tasks with feedback, such as driving to a goal.

NVIDIA Container Toolkit

Lets Docker containers use the host's NVIDIA GPU.

Partition

A section of a drive with its own file system, such as `/`, `/home` or `swap`.

Plymouth

The program that draws the logo and spinner while Ubuntu starts.

Repository (package source)

A server apt installs from. Only add ones from a project's official documentation.

ROS 2

Robot Operating System 2, the standard framework for robotics software. Jazzy Jalisco is the release for Ubuntu 24.04.

roscdep

A ROS tool that installs the system packages listed in each package's `package.xml`.

RViz

The ROS 2 3D viewer for maps, sensor data, robot poses and plans.

SSH key

A pair of files that proves your identity to GitHub without a password. Share only the `.pub` file.

sudo

Runs one command with administrator rights. It asks for your password, which stays invisible as you type.

Swap

Disk space Linux uses as overflow when RAM is full.

systemd

The system that starts Ubuntu's services at boot. `systemd-analyze` measures how long that takes.

Timeshift snapshot

A restore point of the Ubuntu system you can roll back to if an install goes wrong.

TurtleBot3

A small open-source research robot widely used in ROS tutorials and simulation.

Virtual environment (venv)

A private folder of Python packages, so one project's versions cannot break another's.

Sources

The versions and package names in this handbook were checked against these official pages in October 2026. Check them again before you install, because releases change quickly.

Isaac Sim system requirements

<https://docs.isaacsim.omniverse.nvidia.com/latest/installation/requirements.html>

Isaac Sim installation with pip

https://docs.isaacsim.omniverse.nvidia.com/6.1.0/installation/install_python.html

ROS 2 Jazzy installation (Ubuntu, deb packages)

<https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>

Docker Engine installation on Ubuntu

<https://docs.docker.com/engine/install/ubuntu/>

NVIDIA Container Toolkit installation guide

<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

GRUB themes: Tela, Vimix, Stylish and WhiteSur

<https://github.com/vinceliuice/grub2-themes>

ABOUT THE AUTHOR

Md Asifuzzaman focuses on motion planning for robotics and autonomous vehicles. He works with ROS 2, C++ and Nav2, and wrote this handbook from his own workstation setup so that others can reach the real work faster.

Portfolio	asif-ucchwas.github.io
GitHub	github.com/Asif-Ucchwas
LinkedIn	linkedin.com/in/masifuzzaman
Newsletter	mdasifuzzaman.substack.com

THE ROBOTICS WORKSTATION HANDBOOK

Before you can plan a path, fuse a sensor or drive a car, you need a machine that works.

This handbook takes you from a Windows PC to a GPU-ready Ubuntu 24.04 workstation with Docker, ROS 2 Jazzy, Nav2, Gazebo and NVIDIA Isaac Sim. Every command is in order, every chapter ends with a check, and every skill is linked to the engineering roles that need it.

16

CHAPTERS

57

COMMAND BLOCKS

6

ENGINEERING ROLES

For aspiring motion planning, perception, autonomy, simulation, controls and robotics software engineers.